

Escuela Politécnica Nacional

Proyecto de 1er Bimestre

Sistemas de Recuperación de Información

Oscar Albán – Henry Ramirez

10 de noviembre de 2025

1. Descripción del corpus utilizado

El corpus utilizado corresponde a un conjunto de reseñas de videojuegos en la plataforma Steam. Para el proyecto seleccionamos un subconjunto de 20.000, estructuradas de la siguiente forma:

- review_id: identificador único
- text: contenido textual escrito por el usuario
- game: videojuego al que pertenece la reseña
- recommendation: indica si el usuario recomienda o no el juego
- metadata adicional: horas jugadas, fecha y puntaje (cuando está disponible)

Las reseñas tienen características típicas de texto humano: lenguaje informal, variabilidad de idiomas, aunque se seleccionó las reseñas en inglés, abreviaturas y términos compuestos: “open world”, “level design”, expresiones, alto ruido no textual (cadenas repetidas, puntuación excesiva y palabras inventadas). Por eso requerimos un preprocesamiento detallado que controle esto.

2. Explicación de las decisiones de diseño

El sistema se implementó de una manera modular, priorizando la claridad, eficiencia y entendimiento al momento de evaluar.

2.1 Preprocesamiento e índice invertido

Decisiones principales para limpiar el texto y generar los tokens limpios:

- Normalización del texto: minúsculas, eliminación de HTML y puntuación.
- Tokenización y stopwords: el sistema elimina palabras no informativas.
- Eliminación de tokens artificiales: se descartaron cadenas como aaaa, típicas de ruido en reseñas.
- Stemming (Porter): reduce variantes morfológicas y mejora la coincidencia de términos.
- Generación de bigrams: se incorporaron combinaciones como *open_world*, *early_access*, que son cruciales en el dominio de videojuegos. Esta decisión incrementó la calidad de recuperación para términos compuestos.

A partir del corpus preprocesado, donde cada documento tiene una lista de tokens limpios, se reorganizó la información, para crear una estructura que permita identificar rápidamente en qué documentos aparece cada término. Primero expandiendo los tokens en filas individuales para poder calcular la frecuencia de cada término por documento. Con estos datos se construyó el índice invertido: el diccionario donde cada término se asocia a una lista de pares (doc_id, frecuencia). Permitiendo acceder directamente a los documentos en los que aparece cada término.

2.2 Modelos de Recuperación de Información

2.2.1 Modelo basado en Jaccard

Este modelo, utiliza el coeficiente de Jaccard para medir la similitud entre la consulta y cada uno de los documentos, es por ello, que aquí cada documento se representa como un conjunto de términos, ignorando las frecuencias. Esta similitud se calcula con:

$$\frac{\text{tamaño de la intersección}}{\text{tamaño de la unión entre ambos conjuntos}}$$

Este modelo permite obtener un valor referencial rápido de relevancia basado en la presencia o ausencia de términos. Bastante útil cuando se usa como punto de comparación, aunque no calcula la importancia ni la repetición de los términos en los documentos.

2.2.2 Modelo TF-IDF

Este modelo corresponde al clásico TF-IDF con similitud coseno. La consulta se transforma en un vector donde cada uno de los pesos combina la frecuencia del término en la consulta TF con la información inversa de la frecuencia en el corpus IDF. Para cada uno de los documentos también se consulta el peso TF-IDF de los términos que coinciden en la consulta.

La puntuación final se obtiene mediante el producto punto entre el vector de la consulta y el vector del documento, normalizándolos. Este modelo incorpora información sobre la relevancia de los términos en el corpus y la frecuencia dentro del documento, mejorando la discriminación.

2.2.3 Modelo BM25

Uno de los algoritmos más utilizados en los sistemas de recuperación de la información modernos. BM25 pondera cada término combinando su IDF, la frecuencia del término en el documento (TF), y un factor de normalización que contiene la longitud del documento, lo cual controla que los documentos demasiado extensos no dominen en el ranking.

Utiliza parámetros ajustables (k_1 y b) que controlan el grado de saturación del término y que tanto influye la longitud del documento. En nuestro proyecto, usamos una versión optimizada, que solo evalúa los documentos que contienen términos de la consulta, mejorando la eficiencia del modelo. Ofreciéndonos, la mejor calidad de ranking, equilibrando adecuadamente la frecuencia del término, su rareza en el corpus y la longitud del documento.

2.3 Interfaz CLI

La implementación de la interfaz de línea de comandos del sistema tiene como función principal permitir que el usuario interactúe con el índice, ejecutar consultas y visualizar tanto resultados como métricas de evaluación.

El CLI realiza las siguientes tareas:

2.3.1 Carga y preparación del corpus y construcción del índice invertido

- Lee el archivo CSV con las reseñas de Steam.
- Selecciona los primeros 20 000 documentos.
- Aplica preprocesamiento (tokenización, limpieza y normalización).
- Construye una columna tokens para cada documento.
- Llama al módulo de preprocesamiento para generar el índice que luego usan los modelos Jaccard, TF-IDF y BM25.

2.3.2 Interfaz interactiva

- El usuario puede escribir consultas libres en texto.
- El sistema muestra los **top 10** resultados usando cada modelo de recuperación.
- Para cada documento recuperado se presenta una previsualización del texto original.

2.3.3 Módulo de evaluación integrado

- Con el comando evaluar, el CLI ejecuta automáticamente tres consultas de prueba.
- Calcula Precision@10, Recall@10 y AP para cada modelo.
- Finalmente imprime el **MAP** de Jaccard, TF-IDF y BM25.

2.3.4 Control del sistema

- El usuario puede escribir salir para finalizar el programa.
- Todo el flujo es continuo y no requiere reinicio del sistema.

2.4 Sistema de Evaluación

El sistema de evaluación implementa las métricas clásicas de Recuperación de Información para medir la calidad de los resultados obtenidos por el modelo. Se utilizan tres consultas de prueba y un archivo qrels.json que contiene los identificadores de los documentos marcados manualmente como relevantes.

Las métricas implementadas son:

- **Precision**: mide qué proporción de los documentos recuperados son relevantes.
- **Recall**: indica qué porcentaje de los documentos relevantes fueron recuperados por el sistema.
- **Average Precision (AP)**: evalúa si los documentos relevantes aparecen en posiciones altas dentro del ranking, calculando la precisión cada vez que se encuentra un acierto.
- **Mean Average Precision (MAP)**: se obtiene promediando los valores AP de todas las consultas de prueba; es la métrica final del sistema.

Para cada consulta, el sistema:

1. Preprocesa el texto de la consulta.
2. Recupera los **20 documentos mejor rankeados** usando el modelo BM25.
3. Compara estos resultados contra los documentos relevantes definidos en qrels.json.
4. Calcula Precision, Recall y AP.
5. Después de procesar las tres consultas, calcula el **MAP**, que resume las métricas y el rendimiento general del sistema.

Este módulo permite validar objetivamente la calidad de los modelos de recuperación y comparar sus desempeños entre diferentes configuraciones.

3. Ejemplos de consultas y resultados

Al ser un corpus en inglés y basado en reseñas de videojuegos de Steam, las primeras consultas fueron hechas a partir de esta temática, con el fin de verificar que los resultados sean correspondientes a la consulta. A continuación, se presenta ejemplos de consultas con los distintos modelos de RI.

3.1 Consulta: “Good game” con Jaccard

Al ser consulta muy simple y que se repite mucho en las reseñas de videojuegos, el score es de 1 en los primeros documentos a pesar de tener otras palabras como “very” que fueron preprocesadas y eliminadas como stopwords por lo que el score no se ve afectado, la limitación principal del modelo es que no evalúa la importancia de los términos ni su repetición.

```
1. Documento 25285528 | Score=1.0000
   Texto: very good game
2. Documento 12379178 | Score=1.0000
   Texto: Good Game
3. Documento 38579454 | Score=1.0000
   Texto: good game
```

3.2 Consulta: “Counter Strike” con TF-IDF

Este query se ejecutó con el nombre de un videojuego muy conocido y se eligió este modelo como ejemplo ya que el modelo anterior verifica solo la existencia de las palabras de la query dentro de los documentos, motivo por el cuál, los documentos recuperados contienen cualquiera de las palabras, sin necesidad de que sean tomados en cuenta como palabras compuestas o n-gramas, a continuación, se muestra el ranking con TF-IDF

```
TOP 10 TF-IDF
1. Documento 42397692 | Score=1.0000
   Texto: Counter Strike Source is the forgotten brother of counter strike games today but back in its hayday it was an amazing game. anyone wh
   was a fan of valve games were blown away at how they had released...
2. Documento 49599099 | Score=1.0000
   Texto: This version of counter strike is really the bastard child of the counter strike franchise. it came out at a time when source was out
   (or around the corner depending on where you were) and while cs 1....
3. Documento 62933297 | Score=1.0000
   Texto: Game #69
   3.2.2020. [h1] I hate this game and the reason
   for my hatred have nothing to do with the game itself [/h1] Now, sure, I haven't actually played enough of this game to give it a pr...
```

3.3 Consulta: “tactical shooter” con BM25

Para el tercer ejemplo se utilizó una consulta que no aparece directamente en los documentos del corpus, sin embargo, posee términos que suelen ser comunes y a diferencia de otras búsquedas mas exactas el modelo de BM25 no lo hizo por similitud sino por probabilidad, obteniendo los siguientes resultados:

```
TOP 10 BM25
1. Documento 127376169 | Score=7.5594
   Texto: It's just a shooter
2. Documento 106676497 | Score=6.9384
   Texto: boring shooter for boomers
3. Documento 142426942 | Score=6.7461
   Texto: i dont like it i hate multiplayer shooters well its cheap but its same thing to throw ur money into lava but
   or guys who like multiplayer shooters i would recomend this but theres a lot of better sho...
```

4. Análisis de métricas de evaluación

El Sistema de Evaluación implementado en el módulo evaluacion.py tiene como objetivo medir la calidad y efectividad del *ranking* generado por los modelos de recuperación, contrastando los resultados obtenidos con un conjunto de respuestas correctas predefinidas

El módulo utiliza tres consultas de prueba y un archivo “qrels.json” que contiene los identificadores de los documentos marcados manualmente como relevantes

4.4.1 Métricas de Conjunto

Estas métricas miden si los documentos correctos fueron recuperados, sin enfocarse en la posición que ocupan en el ranking.

- **Precision (P):** Mide la proporción de los documentos recuperados que son relevantes. Se calcula en el CLI como Precision@10.

- **Recall (R):** Indica qué porcentaje de los documentos relevantes totales fueron recuperados por el sistema. Se calcula en el CLI como Recall@10.

4.4.2 Métricas de Ranking

Estas métricas son esenciales para evaluar la **calidad del orden** de los resultados, premiando aquellos sistemas que colocan los documentos relevantes en las primeras posiciones⁸.

- **Average Precision AP:** Evalúa si los documentos relevantes aparecen en posiciones altas dentro del ranking. Esto lo hace calculando la precisión cada vez que se encuentra un acierto.
- **Mean Average Precision MAP:** Es la métrica final del sistema. Se obtiene promediando los valores AP de todas las consultas de prueba. El MAP resume el rendimiento general del ranking del sistema y es la métrica estándar para comparar modelos de recuperación.

Proceso de Evaluación en “evaluacion.py”

El módulo automatiza la validación del rendimiento del sistema:

1. El texto de la consulta se preprocesa.
2. Se recuperan los 20 documentos mejor rankeados utilizando el modelo BM25.
3. Estos resultados se comparan contra los documentos relevantes definidos en “qrels.json”.
4. Se calcula, P, R, AP
5. Finalmente, se calcula el, que es impreso como resumen del rendimiento general del sistema. Este módulo permite validar objetivamente la calidad de los modelos de recuperación.

A continuación, se presenta un ejemplo:

```
EJECUTANDO EVALUACION
>>> Query q1: "call of duty"

Modelo: JACCARD
Precision@10: 0.3000
Recall@10:    0.4286
AP:          0.4286

Modelo: TFIDF
Precision@10: 0.1000
Recall@10:    0.1429
AP:          0.0357

Modelo: BM25
Precision@10: 0.4000
Recall@10:    0.5714
AP:          0.5714
```

El análisis de las métricas de evaluación confirma que el modelo BM25 es el algoritmo de ranking más efectivo y preciso del sistema. Su superioridad reside en su capacidad para maximizar la recuperación de documentos relevantes y posicionarlos correctamente en la cima de los resultados.

1. **Máxima Exhaustividad (Recall):** BM25 logró recuperar la mayor cantidad de los documentos relevantes definidos de manera manual y aleatoria, en la verdad fundamental para esa consulta, superando claramente a los modelos Jaccard y TF-IDF.
2. **Alta Calidad de Ranking (AP):** El Average Precision (AP) de BM25 fue el más alto, lo cual confirma que los documentos relevantes no solo fueron encontrados, sino que también fueron posicionados consistentemente en los primeros lugares del ranking, cumpliendo el objetivo principal de un SRI.

En resumen, la capacidad del modelo BM25 para ponderar la frecuencia del término (TF) y normalizar la longitud del documento lo hace inherentemente más robusto que los modelos TF-IDF y Jaccard. Por lo tanto, se establece que BM25 es el algoritmo más preciso para el ranking dentro de este sistema.